Ejercicio 3 — Una pila con `minimo` en $O(1)$

Dificultad: ★ **Conferencias:** C3 (Correctitud)

Enunciado

> *Una pila con una operación `minimo` que cuesta $O(1)$ en el peor caso. Diseña la estructura, enuncia el invariante que la hace correcta y da el costo de cada operación.*

1 El problema

Una pila soporta `push(x)`, `pop()` y ahora también `minimo()`, que debe devolver el menor elemento actualmente en la pila. La solución ingenua — recorrer la pila entera cada vez que se pide el mínimo — cuesta $O(n)$ por consulta. Queremos las tres operaciones en $O(1)$ peor caso, sin amortizar nada (a diferencia de la conferencia 4, aquí *cada* operación individual tiene que ser barata, no solo el promedio de una secuencia).

Especificación (C3 §1). Precondición: `pop()` y `minimo()` se llaman solo sobre una pila no vacía (igual que en cualquier pila estándar, donde `pop` de una pila vacía no está definido). Postcondición: `minimo()` devuelve el menor elemento entre los que están actualmente apilados, sin modificar la pila.

2 Diseño: dos pilas paralelas

La idea es mantener, junto a la pila principal, una segunda pila que en todo momento tiene en su tope el mínimo de lo que hay apilado hasta ahí.

El código real y ejecutable vive en `verif.py`, en esta carpeta; aquí va el pseudocódigo:

```
estructura PilaMin:
    principal ← pila vacia
    minimos   ← pila vacia     // minimos.cima = minimo de todo lo que hay en principal

    funcion push(x):
        principal.apilar(x)
        si minimos vacia o x ≤ minimos.cima:
            minimos.apilar(x)
        sino:
            minimos.apilar(minimos.cima)     // repite el minimo anterior

    funcion pop():
        minimos.desapilar()
        devolver principal.desapilar()

    funcion minimo():
        devolver minimos.cima
```

La tabla siguiente traza un ejemplo concreto, para que se vea cómo las dos pilas evolucionan juntas (no es la demostración — esa es la sección 3 — solo una corrida ilustrativa):

Operación	principal	minimos
`push(5)`	$[5]$	$[5]$
`push(3)`	$[5, 3]$	$[5, 3]$
`push(4)`	$[5, 3, 4]$	$[5, 3, 3]$
`push(1)`	$[5, 3, 4, 1]$	$[5, 3, 3, 1]$
`pop()`	$[5, 3, 4]$	$[5, 3, 3]$
`push(2)`	$[5, 3, 4, 2]$	$[5, 3, 3, 2]$

En cada fila, el último valor de `minimos` es exactamente el mínimo de todo `principal` en ese momento — es el invariante de la sección siguiente, en acción.

3 Invariante

Invariante

Después de cualquier secuencia de operaciones, `principal` y `minimos` tienen la misma longitud y, para todo i: `minimos[i]` $=$ mín(`principal[0..i]`).

A diferencia del invariante de un ciclo (que se demuestra sobre el índice de la vuelta), este es un invariante **sobre la secuencia de operaciones**: se demuestra por inducción en el número de operaciones realizadas. Es del mismo tipo que los dos invariantes de Gale-Shapley de C3 §7 ("un receptor nunca vuelve a quedar libre", "un proponente solo empeora"), que también son afirmaciones sobre el estado después de cualquier prefijo de ejecución, no sobre un ciclo con contador.

Inicialización. Pila vacía: ambas listas vacías (misma longitud, 0), la afirmación es vacuamente cierta.

Mantenimiento. Supongamos el invariante cierto antes de una operación.

- `push(x)`: las posiciones $0, \ldots, i-1$ de ambas pilas no se tocan, así que siguen cumpliendo el invariante por hipótesis, y ambas pilas crecen una posición, así que siguen con la misma longitud. Falta verificar la posición nueva, i (donde queda x). Si $i = 0$ la pila estaba vacía: el código guarda x en `minimos[0]`, y mín(`principal[0..0]`) $= x$. Si $i \geq 1$, por hipótesis, `minimos[i-1]` $=$ mín(`principal[0..i-1]`). El código compara x con ese valor y guarda el menor de los dos en `minimos[i]`, que es exactamente mín(`principal[0..i-1]` $\cup \{x\}$) $=$ mín(`principal[0..i]`).
- `pop()`: se elimina la última posición de ambas pilas a la vez (siguen con la misma longitud). El invariante para las posiciones restantes $0, \ldots, i-1$ no cambió (no se tocaron esos valores), así que sigue valiendo.

Conclusión. En cualquier momento con la pila no vacía (la precondición de la sección 1), por el invariante aplicado al último índice, `minimos[-1]` $=$ mín(`principal`), que es justo lo que `minimo()` devuelve. Esto da la **correctitud parcial**.

Terminación. A diferencia de un algoritmo con un ciclo (donde hace falta un potencial que baje), aquí no hay nada que demostrar: `push`, `pop` y `minimo` son código lineal, sin ciclos ni recursión, así que cada uno termina trivialmente en una cantidad fija de pasos. Correctitud parcial más terminación dan la **correctitud total** (C3 §1).

4 Costo

Operación	Tiempo	Por qué
`push`	$O(1)$	una comparación y dos operaciones de apilado
`pop`	$O(1)$	dos operaciones de desapilado del tope
`minimo`	$O(1)$	un acceso al tope

Modelo para el peor caso. La garantía estricta de $O(1)$ supone que las pilas subyacentes implementan apilar, desapilar y consultar el tope en $O(1)$ peor caso; por ejemplo, mediante nodos enlazados, o con un arreglo cuya capacidad ya esté garantizada y que no se redimensione durante la operación. `verif.py` usa listas de Python solo para comprobar la correctitud funcional: en un arreglo dinámico, `append` y una posible reducción de capacidad pueden ser amortizados, así que esa implementación de prueba no demuestra por sí sola la cota de peor caso.

Memoria: $\Theta(n)$ adicional en el peor caso, con n el número de elementos actualmente apilados — la segunda pila `minimos`, del mismo tamaño que la pila principal.

Verificación. La demostración de la sección 3 ya garantiza que `minimo()` es correcto; se corrió de todas formas como chequeo empírico adicional (en `verif.py`), comparando contra un oráculo de fuerza bruta (`min(lista)` sobre una lista de Python que refleja el estado real) sobre 2000 secuencias aleatorias de `push`/`pop`, consultando `minimo()` después de cada operación:

```
PilaMin (dos pilas): 2000 secuencias aleatorias verificadas, sin fallos
```

Respuesta a la pregunta del enunciado

- **Estructura:** dos pilas paralelas, `principal` y `minimos`, donde `minimos` guarda en cada posición el mínimo de `principal` hasta ahí (sección 2).
- **Invariante:** ambas pilas de la misma longitud y `minimos[i]` $=$ mín(`principal[0..i]`) para todo i, demostrado por inducción sobre la secuencia de operaciones, no sobre un índice de ciclo (sección 3).
- **Costo:** `push`, `pop` y `minimo` son los tres $O(1)$ peor caso (no amortizado, bajo el modelo de implementación de la sección 4); memoria $\Theta(n)$ adicional (sección 4).

Errores que hay que vigilar

- **Usar `<` en vez de `<=` al comparar `x` con el mínimo actual en `push`.** Con `<` estricto, cuando se apila un valor **igual** al mínimo actual, el código no lo agregaría de nuevo a `minimos` con su propia entrada — pero como aquí *siempre* se agrega una entrada por cada `push` (se repite el valor anterior si no hay nuevo mínimo), este error concreto no rompe esta implementación particular. El error real aparece en variantes que **solo** apilan en `minimos` cuando hay un mínimo *estricto* (para ahorrar memoria, guardando pares `(valor, repeticiones)` en vez de una entrada por cada `push`): ahí, si se usa `<` en vez de `<=` al decidir si el valor nuevo iguala al mínimo vigente, un `pop()` puede descontar una repetición de más y sacar el mínimo de la pila de mínimos cuando en realidad todavía queda una copia de ese valor en la pila principal. El arreglo es tratar la igualdad como "suma una repetición más", no como "ignóralo".
- **Olvidar que `pop()` tiene que sacar de *las dos* pilas a la vez.** Si solo se saca de `principal`, el invariante se rompe de inmediato: `minimos` queda con una entrada de más, desalineada con `principal`, y `minimo()` empieza a devolver el mínimo de un conjunto que ya no es el que está apilado.
- **Pensar que una sola pila de este tipo resuelve "mínimo de una cola".** Con una pila, deshacer la última operación (`pop`) es barato porque es simétrico a `push`. Con una **cola** (FIFO) no hay esa simetría, y el problema de mantener el mínimo (o el máximo) con `encolar`/`desencolar` en $O(1)$ amortizado es genuinamente más difícil — se resuelve con una **cola doble monótona** (un `deque` donde se van descartando desde atrás los elementos que nunca podrán ser el mínimo porque hay uno más chico y más reciente), o con dos pilas `PilaMin` haciendo de cola (la de C4 §2), también $O(1)$ amortizado.